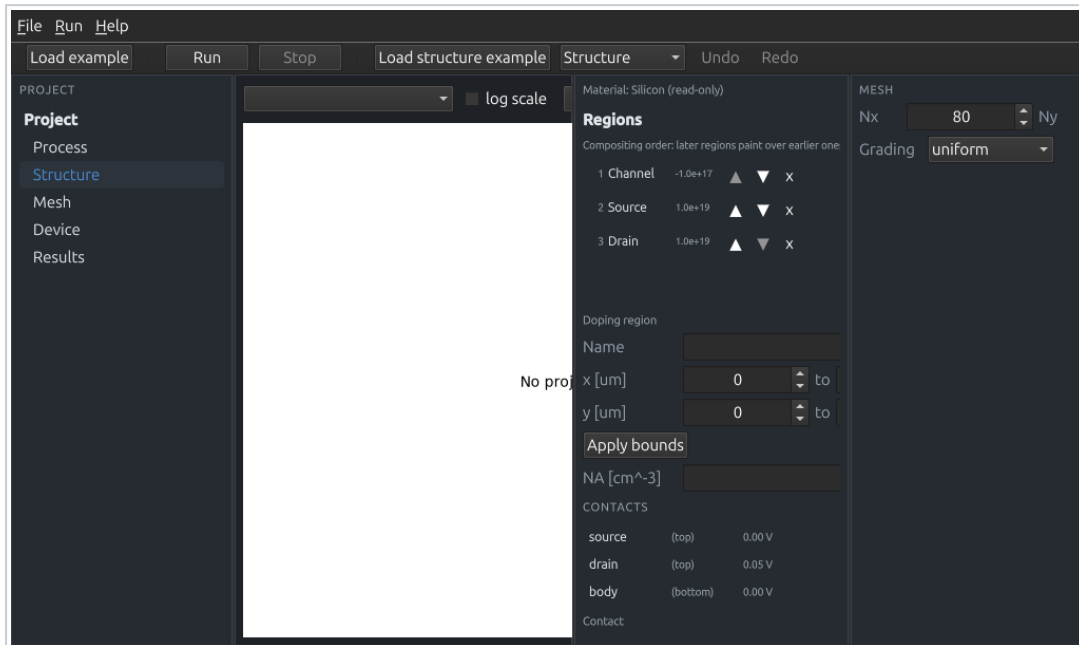


# PyTCAD Desktop GUI

User Guide — v0.1 + v0.2 (Structure & Mesh Workbench)

A practical, task-based guide to loading, editing, solving, and saving 2D semiconductor device structures in the PyTCAD desktop application.



*The Structure + Mesh Workbench, running on a real display.*

# Contents

1. Installing & Launching
2. Interface Overview
3. Loading a Structure
4. Editing Regions
5. Contacts & Gates
6. Mesh Configuration
7. Validation
8. Running a Simulation & Viewing Results
9. Undo / Redo
10. Saving & Loading Projects
11. Keyboard Shortcuts
12. Known Limitations
13. Troubleshooting

# 1. Installing & Launching

Requires Python 3.9+ and PySide6 6.10.1 or later. Run these commands from the **pytcad** project directory:

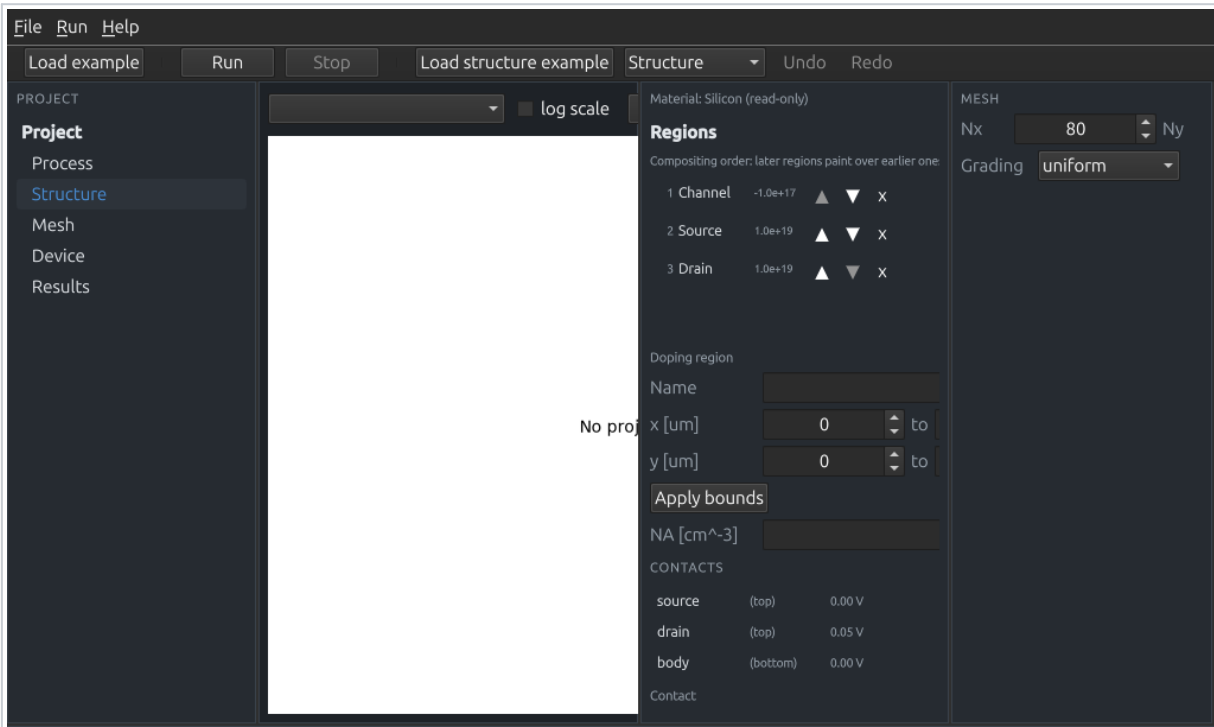
```
pip install -r requirements.txt  
pip install -r gui/requirements.txt
```

Then launch the application:

```
python -m gui.app
```

This opens the main window directly on your desktop — it is a native PySide6/Qt Quick application, not a web page.

## 2. Interface Overview



*The main window with a structure loaded.*

The window is divided into seven areas:

- **Project Tree** (far left) — navigation for Process / Structure / Mesh / Device / Results. Click a node to see its properties in the Properties panel.
- **Toolbar** (top) — Load example, Run, Stop, Load structure example, the Structure/Doping/Mesh/Results view-mode selector, and Undo/Redo.
- **Viewport** (center) — the plot area. Shows the doping map, the structure diagram, the mesh grid, or solved results, depending on the selected view mode.
- **Structure panel** (center-right) — Material, Regions, the Doping editor, Contacts, Gates, and Validation, all for the currently loaded structure.
- **Mesh panel** (right) — Nx/Ny, grading mode, and a live node/cell/memory estimate.
- **Properties panel** (far right) — read-only details for whatever is selected in the Project Tree.
- **Simulation Console** (bottom) — a running log of everything the app does: loads, saves, solver progress, and errors.

### 3. Loading a Structure

PyTCAD ships two kinds of built-in example, reached from the **File** menu or the toolbar:

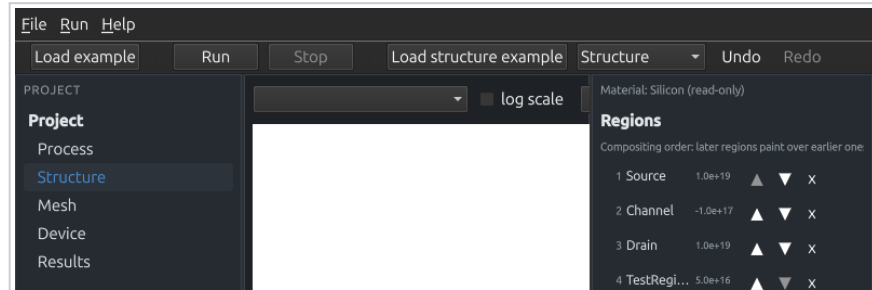
- **Load example** ("mosfet\_2d") — a 2D MOSFET with a smooth, analytic doping profile. This is the fastest way to see a solved result, but it is **not editable** through the Structure panel.
- **Load structure example** ("mosfet\_2d\_structure") — a second, rectangular-region MOSFET built specifically for this workbench. Regions, contacts, gates, and the mesh are all fully editable.

These are deliberately two separate examples rather than one: the analytic profile of the first cannot be losslessly represented as editable rectangular regions.

Click **Load structure example** to follow the rest of this guide — every other section assumes a structure is loaded.

## 4. Editing Regions

A **region** is a rectangular area of uniform doping. The Regions list shows every region in the current structure, in **compositing order**: later regions paint over earlier ones wherever their rectangles overlap. The number on the left of each row is its priority rank.



*The Regions list — priority, name, doping, and reorder/remove controls.*

**To add a region:** click the + button above the list. A new region is created with default (tiny) bounds and doping — select it and edit its fields below.

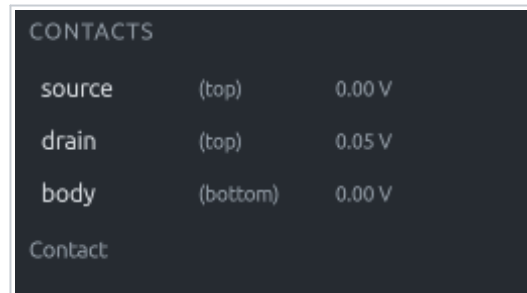
**To edit a region:** click its row to select it. Its Name, x/y bounds, and net doping appear in the Doping region editor beneath the list. Edit a field and press Enter/Tab, or click **Apply bounds** for the x/y spin boxes.

**To reorder or remove a region:** use the ▲ / ▼ buttons on its row to change its compositing priority, or X to delete it. All three are undoable.

Net doping is a single signed number: positive = donor ( $N_D$ , n-type), negative = acceptor ( $N_A$ , p-type). The field label switches automatically based on the sign you enter.

## 5. Contacts & Gates

Below the Regions section, the Structure panel lists every **contact** and **gate** on the structure — each is a boundary condition on one edge of the domain.



| CONTACTS |          |        |
|----------|----------|--------|
| source   | (top)    | 0.00 V |
| drain    | (top)    | 0.05 V |
| body     | (bottom) | 0.00 V |

Contact

*The Structure panel's Contacts section (name, edge, voltage).*

**Contacts** are simple ohmic terminals: click a row to select it, then set its voltage in the field below. Each row shows the contact's name, which edge it sits on, and its current voltage.

**Gates** have more to configure: tox (oxide thickness), gate voltage, and flatband voltage mode.

- **Computed** (default) —  $V_{fb}$  is calculated from the substrate doping directly beneath the gate, using the same physics as the built-in MOSFET model. The gate's footprint must sit over a single, uniformly-doped region — if it spans two differently-doped regions, validation will flag it (see §7).
- **Manual** — enter  $V_{fb}$  directly, for calibration against measured devices.

Material is fixed to Silicon and shown read-only at the top of the panel — the underlying solver supports one semiconductor material per structure.

## 6. Mesh Configuration

The Mesh panel (top right) controls the simulation grid.

- **Nx / Ny** — node counts along x and y. Click Apply after changing either value.
- **Grading** — *uniform* spacing, or *graded* (finer near the domain center, coarser toward the edges) via the existing graded-mesh utility.
- The list below shows live node/cell counts, min/max spacing, and a rough memory estimate for the current mesh — watch this before committing to a very fine grid.

A mesh that is too coarse near a junction will silently produce an inaccurate solve; a mesh that is too fine can take a long time or use a lot of memory. Use the memory estimate as a sanity check, not a guarantee.



## 7. Validation

The Structure panel's VALIDATION section lists every problem with the current structure — it must be empty (showing **OK**) before you can Run or Save. Common issues it catches:

- Non-positive domain width/height, or a mesh with fewer than 2 nodes in either direction.
- A region with zero/negative width or height, or one that extends outside the domain.
- Duplicate region, contact, or gate names.
- A gate with  $t_{ox} \leq 0$ .
- A gate in manual  $V_{fb}$  mode with no value entered.
- A gate (in computed mode) whose footprint spans regions with different doping — the message reports the doping range found, so you know how to narrow the gate or switch it to manual mode.

## 8. Running a Simulation & Viewing Results

Click **Run** in the toolbar. If a structure is loaded, PyTCAD validates it, converts it to a solver job, and runs the drift-diffusion solve in a **separate process** — the window stays responsive, and Newton-iteration progress streams into the Simulation Console as it happens.

Click **Stop** to cancel a running solve. Because results are written atomically, a canceled run leaves nothing behind — you will never see a half-written result.

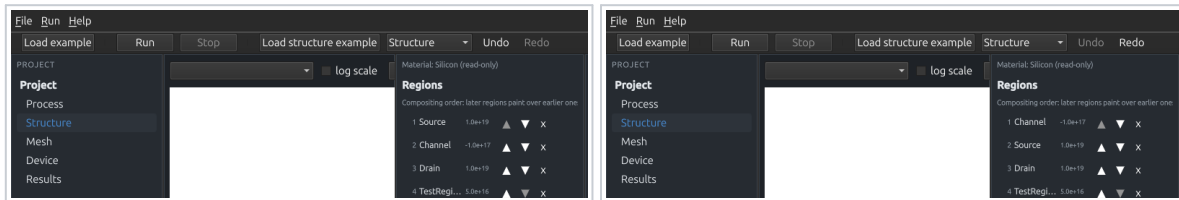
Once a solve finishes, use the view-mode dropdown next to Run to switch the viewport:

- **Structure** — a diagram of regions, contacts, and gates (colored by doping sign).
- **Doping** — the solved doping field as a heatmap.
- **Mesh** — the grid lines themselves.
- **Results** — potential, electron density, hole density, or terminal currents, with zoom / pan / fit / reset controls and an optional log-scale toggle.

The Structure and Mesh diagram views have a known display issue in this build — the diagram can render zoomed in further than intended. The underlying data and any solved Results plots are unaffected; this is a cosmetic viewport issue under investigation.

## 9. Undo / Redo

Every structure and mesh edit — adding, editing, reordering, or removing a region; changing a contact/gate; changing Nx/Ny or grading — goes through a single undo stack. Use the **Undo / Redo** buttons in the toolbar, or **Ctrl+Z / Ctrl+Y**.



*Before Undo (Undo enabled, region order changed) — After Undo (order reverted, Redo now enabled).*

The window title shows an asterisk (**PyTCAD \***) whenever there are unsaved changes. It disappears exactly when the undo stack returns to the last-saved state — even if that means undoing past several edits and back again.

## 10. Saving & Loading Projects

Open **File** → **Save Project...** or **File** → **Load Project...** (or press **Ctrl+S** for Save) to open the project dialog. Enter a file path (e.g. `/tmp/my_project.json`) and, for Save, a project name.

This is a path-entry dialog, not a native OS file browser — you type or paste the full path. A native file picker is planned for a future version.

Saving requires the structure to pass validation first (§7) — an invalid structure cannot be saved. Project files are schema-versioned; every region, contact, gate, and mesh setting round-trips exactly. Solved results are **never** embedded in the project file — they stay as separate `.npz` files.

Loading a project resets the undo stack — you cannot undo past a fresh load back into the previous project's edit history.

If you try to close the window with unsaved changes, a confirmation dialog offers **Don't Save** or **Cancel** (there is no in-dialog Save yet — save first with Ctrl+S if you want to keep your changes).

## 11. Keyboard Shortcuts

| Action                     | Shortcut / Location              |
|----------------------------|----------------------------------|
| Undo                       | Ctrl+Z                           |
| Redo                       | Ctrl+Y                           |
| Save Project               | Ctrl+S                           |
| Load 2D MOSFET example     | File menu                        |
| Load 2D MOSFET (Structure) | File menu, or toolbar button     |
| Load Project...            | File menu                        |
| Run simulation             | Run menu, or toolbar Run button  |
| Cancel simulation          | Run menu, or toolbar Stop button |
| Quit                       | File menu                        |

## 12. Known Limitations

### Not yet implemented:

- A native OS file picker for Save/Load (currently the path-entry dialog above).
- Per-region doping *profiles* beyond uniform (e.g. Gaussian/erfc shapes).
- A live pre-solve doping preview in the Doping viewport mode.
- A second real semiconductor material.

### Not supported, by design:

- Meshed SiO<sub>2</sub> or metal regions — oxide and metal remain boundary-condition concepts (gate oxide thickness, contact voltage), never separate materials.
- Process simulation, FinFET/GAA structures, 3D structure editing, adaptive mesh refinement, or arbitrary polygon/CAD geometry.

### Cosmetic issues:

- The Structure/Mesh diagram viewport can render more zoomed-in than the full domain (display only — does not affect solved results or saved data).
- The mesh-info list can print a handful of harmless console warnings when its contents change; the displayed values are always correct.

## 13. Troubleshooting

| Symptom                                                                                                                        | Likely cause / fix                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Window won't open, PySide6 import error                                                                                        | Re-run the pip install commands in §1; confirm Python 3.9+.                                     |
| "Cannot run an invalid structure"                                                                                              | Check the VALIDATION list (§7) and fix every listed error, then Run again.                      |
| Run button does nothing                                                                                                        | Load an example first (§3) — Run needs a loaded structure or v0.1 example.                      |
| App won't quit / seems frozen when the close confirmation dialog needs a human to answer it whenever there are unsaved changes | The close confirmation dialog needs a human to answer it whenever there are unsaved changes     |
| A backend error appears                                                                                                        | Errors are shown as a concise message with an expandable traceback, and the app itself will not |

---

For architecture details, the backend/GUI boundary, and the full capability matrix, see `gui/README.md` in the pytcad project.